Camunda - 8 Database schema details

Is there a way to see the C8 DB schema details of execution, ACT, RE,HI, Deployment and variables?

There is no relational database in Camunda 8. Data is exported to Elasticsearch, you can peek into the indices to see the schema - however, the schema differs per App (Optimize/Operate/Tasklist)

Do we have any example for this? Data will be automatically exported to Elasticsearch or have to do and how to do?

You can only access Elasticsearch in Self-Managed Camunda 8. In SaaS, you cannot access Elasticsearch directly and would need to extract the required information via one of the APIs.

If you deployed Camunda 8 Using the Helm Charts, the Elasticsearch exporter is activated automatically. You can use Elasticsearches REST API 71 to get and inspect all indices.

You can also write your own exporter:

Exporters | Camunda Platform 8 Docs **38**

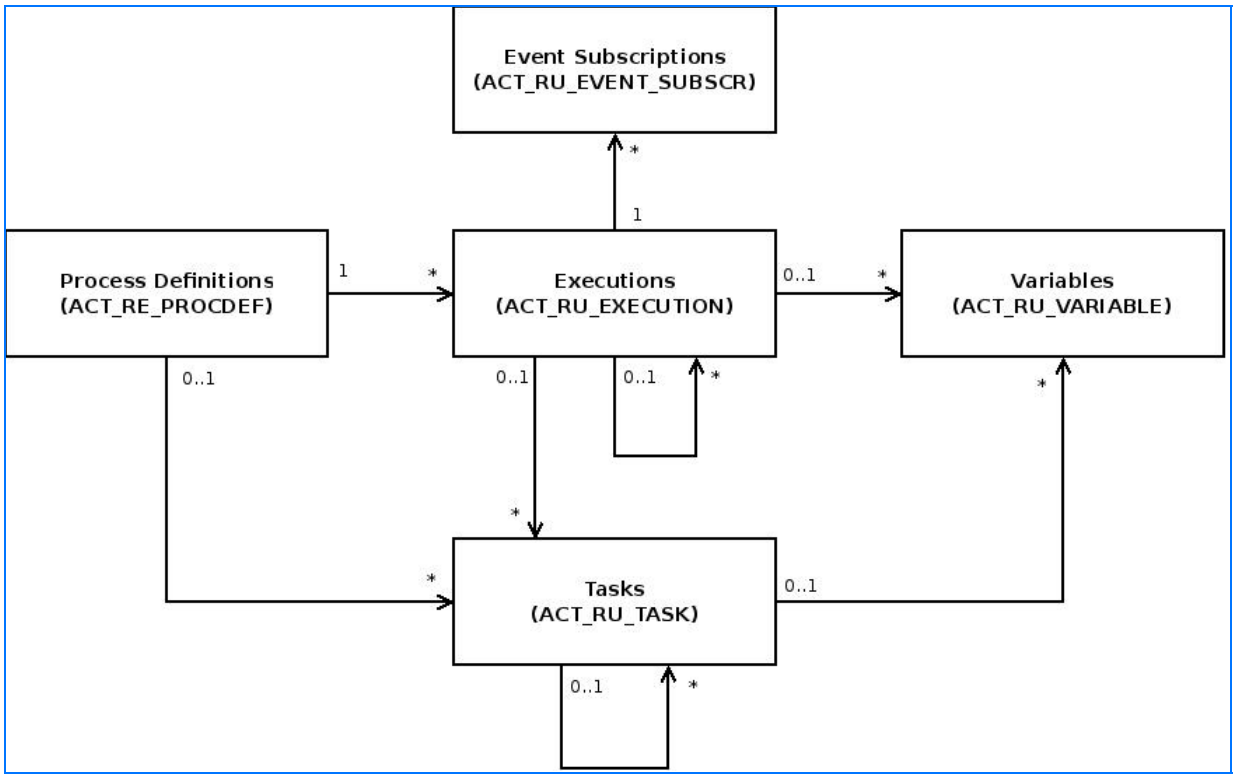
As Zeebe processes jobs and processes, or performs internal maintenance, it generates an ordered stream of records.

Camunda 7 Database Schema

The database schema of the process engine consists of multiple tables. The table names all start with ACT. The second part is a two-character identification of the use case of the table. This use case will also roughly match the service API.

- `ACT_RE_*`: `RE` stands for repository. Tables with this prefix contain 'static' information such as process definitions and process resources (images, rules, etc.).
- `ACT_RU_*`: `RU` stands for runtime. These are the runtime tables that contain the runtime data of process instances, user tasks, variables, jobs, etc. The engine only stores the runtime data during process instance execution and removes the records when a process instance ends. This keeps the runtime tables small and fast.
- `ACT_ID_*`: `ID` stands for identity. These tables contain identity information such as users, groups, etc.
- `ACT_HI_*`: `HI` stands for history. These are the tables that contain historical data such as past process instances, variables, tasks, etc.
- `ACT_GE_*`: General data, which is used in various use cases.

The main tables of the process engines are the entities of process definitions, executions, tasks, variables and event subscriptions. Their relationship is shown in the following UML model.



Process Definitions (`ACT_RE_PROCDEF`)

The `ACT_RE_PROCDEF` table contains all deployed process definitions. It includes information like the version details, the resource name or the suspension state.

Executions (`ACT_RU_EXECUTION`)

The `ACT_RU_EXECUTION` table contains all current executions. It includes information like the process definition, parent execution, business key, the current activity and different metadata about the state of the execution.

Tasks (`ACT_RU_TASK`)

The `ACT_RU_TASK` table contains all open tasks of all running process instances. It includes information like the corresponding process instance, execution and also metadata such as creation time, assignee or due date.

Variables (ACT_RU_VARIABLE)

The `ACT_RU_VARIABLE` table contains all currently set process or task variables. It includes the names, types and values of the variables and information about the corresponding process instance or task.

Event Subscriptions (ACT_RU_EVENT_SUBSCR)

The `ACT_RU_EVENT_SUBSCR` table contains all currently existing event subscriptions. It includes the type, name and configuration of the expected event along with information about the corresponding process instance and execution.

Schema Log (ACT_GE_SCHEMA_LOG)

The `ACT_GE_SCHEMA_LOG` table contains a history of the database schema version. New entries to the table are written when changes to the database schema are made. On database creation the initial entry is added. Every update script adds a new entry containing an `id`, the `version` the database was updated to and the date and time (`timestamp`) of the update.

To retrieve entries from the schema log, the SchemaLogQuery-API can be used:

```
List<SchemaLogEntry> entries =
managementService.createSchemaLogQuery().list();
```

Metrics Log (ACT_RU_METER_LOG)

The `ACT_RU_METER_LOG` table contains a collection of runtime metrics that can help draw conclusions about usage, load and performance of Camunda 7. Metrics are reported as numbers in the Java `long` range and count the occurrence of specific events.

The default configuration of the MetricsReporter will create one row per metric in `ACT_RU_METER_LOG` every 15 minutes.

Heads Up!

If you are an enterprise customer, your license agreement might require you to report some metrics annually. Please store `root-process-instance-start`, `activity-instance-start`, `executed-decision-instances` and `executed-decision-elements` metrics for at least 18 months until they were reported.

Task Metrics Log (ACT_RU_TASK_METER_LOG)

The `ACT_RU_TASK_METER_LOG` table contains a collection of task related metrics that can help draw conclusions about usage, load and performance of the BPM platform. Task metrics contain a pseudonymized and fixed-length value of task assignees and their time of appearance.

Every assignment of a task to an assignee will create one row in `ACT_RU_TASK_METER_LOG`.

Heads Up!

If you are an enterprise customer, your license agreement might require you to report some metrics annually. Please store task metrics for at least 18 months until they were reported.

Camunda 8 Exporters

As Zeebe processes jobs and processes, or performs internal maintenance (e.g. raft failover), it generates an ordered stream of records.

NOTE

Exporters are not available in Camunda 8 Software-as-a-Service (SaaS).

While the clients provide no way to inspect this stream directly, Zeebe can load and configure user code that can process each record in the form of an exporter.

An **exporter** provides a single entry point to process every record written on a stream.

- Persist historical data by pushing it to an external data warehouse.
- Export records to a visualization tool (e.g. zeebe-simple-monitor).

Zeebe only loads exporters configured through the main Zeebe YAML configuration file.

Once an exporter is configured, the next time Zeebe starts, the exporter starts receiving records. Note that it is only guaranteed to see records produced from that point on.

Find a reference implementation in the form of the Zeebe-maintained Elasticsearch exporter.

The main impact exporters have on a Zeebe cluster is that they remove the burden of persisting data indefinitely.

Once data is not needed by Zeebe anymore, it queries its exporters to know if it can be safely deleted, and if so, permanently erases it, thereby reducing disk usage.

NOTE

If no exporters are configured, Zeebe automatically erases data when it is not necessary anymore. If you need historical data, you **must** configure an exporter to stream records into your external data warehouse.

Regardless of how an exporter is loaded (whether through an external JAR or not), all exporters interact in the same way with the broker, which is defined by the exporter interface.

Loading

Once configured, exporters are loaded as part of the broker startup phase, before any processing is done.

During the loading phase, the configuration for each exporter is validated, such that the broker will not start if:

- An exporter ID is not unique
- An exporter points to a non-existent/non-accessible JAR
- An exporter points to a non-existent/non-instantiable class
- An exporter instance throws an exception in its `Exporter#configure` method.

The last point is there to provide individual exporters to perform lightweight validation of their configuration (e.g. fail if missing arguments). For this validation call, the given context will report a partition ID (via `Context#getPartitionId()`) with a pseudo-null value, i.e. `Context#NULL_PARTITION_VALUE`). However, at runtime, it will report the appropriate partition ID on which the exporter is running.

One caveat is that an instance of an exporter is created and immediately thrown away. Therefore, exporters should not perform any computationally heavy work during instantiation/configuration.

Metrics

The exporter is provided with a Micrometer MeterRegistry in the `Exporter#configure(Context)` method through the configuration. Any metrics to be exported should interact with the registry, for example:

```
public class SomeExporter implements Exporter {
    @Override
    public void configure(final Context context) {
        // ...
        registry = context.getMeterRegistry();
        // ...
    }

    public void flush() {
        try (final var ignored = Timer.resource(registry, "meter.name")) {
            exportBulk();
        }
    }
}
```

When an exporter is validated, it is only provided with an in-memory register which is then discarded.

NOTE

Zeebe creates a single isolated class loader for every JAR referenced by exporter configurations. If the same JAR is reused to define different exporters, these will share the same class loader.

Therefore, different exporters can depend on the same third-party libraries without worrying about versions or class name collisions.

Additionally, exporters use the system class loader for system classes, or classes packaged as part of the Zeebe JAR.

Exporter-specific configuration is handled through the exporter's `[exporters.args]` nested map. This provides a `Map<String, Object>` passed directly in the form of a configuration object when the broker calls the `Exporter#configure(Configuration)` method.

Configuration occurs at two different phases: during the broker startup phase, and once every time a leader is elected for a partition.

Processing

At any given point, there is exactly one leader node for a given partition.

Whenever a node becomes the leader for a partition, it runs an instance of an exporter stream processor.

This stream processor creates exactly one instance of each configured exporter, and forwards every record written on the stream to each of these in turn.

NOTE

This implies there will be exactly one instance of every exporter for every partition. If you have four partitions, and at least four threads for processing, there are potentially four instances of your exporter exporting simultaneously.

Zeebe only guarantees at-least-once semantics. That is, a record is seen at least once by an exporter, maybe more. Cases where this may happen include:

- During reprocessing after raft failover (i.e. new leader election)
- On error if the position is not yet updated

To reduce the amount of duplicate records an exporter processes, the stream processor keeps track of the position of the last successfully exported record for every single exporter. The position is sufficient since a stream is an ordered sequence of records whose position is monotonically increasing. This position is set by the exporter once it can guarantee a record is successfully updated.

NOTE

Although Zeebe tries to reduce the amount of duplicate records an exporter must handle, it is likely it will have to. Therefore, it is necessary that export operations be idempotent. This can be implemented either in the exporter itself, but if it exports to an external system, it is recommended you perform deduplication there to reduce the load on Zeebe. Refer to the exporter-specific documentation for how this is meant to be achieved.

Error handling

If an error occurs during the `Exporter#open(Context)` phase, the stream processor fails and is restarted, potentially fixing the error. Worst case scenario, this means no exporter runs until these errors stop.

If an error occurs during the `Exporter#close` phase, it is logged, but will still allow other exporters to gracefully finish their work.

If an error occurs during processing, we continuously retry the same record until no error is produced. Worst case scenario, this means a failing exporter could bring all exporters to a halt. Currently, exporter implementations are expected to implement their own retry/error handling strategies, though this may change in the future.

Performance impact

Zeebe naturally incurs a performance impact for each loaded exporter. A slow exporter slows down all other exporters for a given partition, and in the worst case, could completely block a thread.

It's therefore recommended to keep exporters as simple as possible, and perform any data enrichment or transformation through the external system.

Exporters

Zeebe comes packaged with three exporters:

- Elasticsearch
- OpenSearch
- Camunda Exporter

Install Zeebe exporters

Camunda 8 Self-Managed Helm chart supports the addition of Zeebe exporters by using `initContainer`.

The following is an example to install Zeebe Hazelcast and Kafka exporters.

```
extraInitContainers:
  - name: init-exporters-hazelcast
    image: busybox:1.35
    command: ["/bin/sh", "-c"]
    args:
      [
        "wget --no-check-certificate
https://repo1.maven.org/maven2/io/zeebe/hazelcast/zeebe-hazelcast-
exporter/0.8.0-alpha1/zeebe-hazelcast-exporter-0.8.0-alpha1-jar-with-
dependencies.jar -O /exporters/zeebe-hazelcast-exporter.jar; ls -al
/exporters",
      ]
    volumeMounts:
      - name: exporters
        mountPath: /exporters/
  - name: init-exporters-kafka
```

```
    image: busybox:1.35
    command: ["/bin/sh", "-c"]
    args:
      [
        "wget --no-check-certificate https://github.com/zeebe-io/zeebe-
kafka-exporter/releases/download/1.1.0/zeebe-kafka-exporter-1.1.0-uber.jar
-O /exporters/zeebe-kafka-exporter.jar; ls -al /exporters",
      ]
    volumeMounts:
      - name: exporters
        mountPath: /exporters/
env:
  - name: ZEEBE_BROKER_EXPORTERS_HAZELCAST_JARPATH
    value: /exporters/zeebe-hazelcast-exporter.jar
  - name: ZEEBE_BROKER_EXPORTERS_HAZELCAST_CLASSNAME
    value: io.zeebe.hazelcast.exporter.HazelcastExporter
  - name: ZEEBE_HAZELCAST_REMOTE_ADDRESS
    value: "{{ .Release.Name }}-hazelcast"
```

This example is downloading the exporters' JAR from a URL and adding the JAR to the `exporters` directory, which will be scanned for JARs and added to the Zeebe Broker classpath. Then, with `environment variables`, you can configure the exporter parameters.

NOTE

During startup, the Zeebe pods will copy the exporters located in `/exporters/` to `/usr/local/zeebe/exporters/`. If the pods have a read-only root file system, a writable volume must be mounted to `/usr/local/zeebe/exporters/`.

Camunda Exporter

The Camunda Exporter exports Zeebe records directly to Elasticsearch/OpenSearch. Unlike the Elasticsearch and OpenSearch exporters, records are exported in the format required by Operate and Tasklist, and configuring additional importers or data transformations is not required.

Using the Camunda Exporter can increase process instance throughput, and can reduce the latency of changes appearing in Operate and Tasklist.

NOTE

When exporting, indexes are created as required, and will not be created twice if they already exist. However, once disabled, they will not be deleted (that is up to the administrator.) A retention policy can be configured to automatically delete data after a certain number of days.

Configuration

Enable the exporter by configuring the `className` in your broker configuration:

```
exporters:
  camundaExporter:
    className: io.camunda.exporter.CamundaExporter
    args:
    # Refer to the table below for the available args options
```

NOTE

As the exporter is packaged with Zeebe, it is not necessary to specify a `jarPath`.

Configure the exporter by providing `args`. See the tables below for configuration options and default values, or review the example YAML configuration.

Option	Description	Default
connect	Refer to Connect for the connection configuration options.	
index	Refer to Index for the index configuration options.	
bulk	Refer to Bulk for the bulk configuration options.	
retention	Refer to Retention for the retention configuration options	
createSchema	If `true` missing indexes will be created automatically.	true

Options

- Connect

Option	Description	Default
type	the type of the underlying search engine to export to. Accepted values are `elasticsearch` or `opensearch`.	elasticsearch
clusterName	The name of the Elasticsearch/OpenSearch cluster to export to.	elasticsearch
dateFormat	Defines a custom date format that should be used for fetching date data from the engine (should be the same as in the engine)	yyyy-MM-dd'T'HH:mm:ss.SSSZZ
socketTimeout	Defines the socket timeout in milliseconds, which is the timeout for waiting for data.	
connectTimeout	Determines the timeout in milliseconds until a connection is established.	
username	Username used to authenticate.	

password	Password used to authenticate.	
security	Refer to Security for security configuration.	

Security

Option	Description	Default
enabled	If `true`, enables the security (ssl) features for the exporter.	false
certificatePath	The file path to the SSL certificate used for secure communication with Elasticsearch/OpenSearch.	
verifyHostname	If `true`, the hostname of the SSL certificate will be validated.	true
selfSigned	If `true`, allows the use of self-signed SSL certificates.	false

Index

Option	Description	Default

prefix	This prefix will be appended to every index created by the exporter; must not contain _ (underscore).	
numberOfShards	The number of shards used for each created index.	3
numberOfReplicas	The number of shard replicas used for created index.	0
variableSizeThreshold	Defines a threshold for variable size. Variables exceeding this threshold are split into two properties: `FULL_VALUE` (full content, not indexed) and `VALUE` (truncated content, indexed).	8191
shardsByIndexName	A map where the key is the index name and the value is the number of shards, allowing you to override the default `numberOfShards` setting for specific indices.	
replicasByIndexName	A map where the key is the index name and the value is the number of replicas, allowing you to override the default `numberOfReplicas` setting for specific indices.	

Bulk

To avoid too many expensive requests to the Elasticsearch/OpenSearch cluster, the exporter performs batch updates by default. The size of the batch, along with how often it should be flushed (regardless of size) can be controlled by configuration.

Option	Description	Default

Option	Description	Default
delay	Delay, in seconds, before force flush of the current batch. This ensures that even when we have low traffic of records, we still export every once in a while.	`5`
size	The amount of records a batch should have before we flush the batch.	`1000`

With the default configuration, the exporter will aggregate records and flush them to Elasticsearch/OpenSearch:

1. When it has aggregated 1000 records.
2. Five seconds have elapsed since the last flush (regardless of how many records were aggregated).

Retention

A retention policy can be set up to delete old data. When enabled, this creates an Index Lifecycle Management (ILM) Policy that deletes the data after the specified `minimumAge`. All index templates created by this exporter apply the created ILM Policy.

Option	Description	Default
enabled	If `true` the ILM Policy is created and applied to the index templates.	`false`
minimumAge	Specifies how old the data must be, before the data is	`30d`

Option	Description	Default
	deleted as a duration.	
policyName	The name of the created and applied ILM policy.	

NOTE

The duration can be specified in days `d`, hours `h`, minutes `m`, seconds `s`, milliseconds `ms`, and/or nanoseconds `nanos`.

Example

Here is an example configuration of the exporter:

```
---
exporters:
  # Camunda Exporter ----------
  # An example configuration for the camunda exporter:
  #
  # These setting can also be overridden using the environment variables
"ZEEBE_BROKER_EXPORTERS_CAMUNDAEXPORTER_..."
  # To convert a YAML formatted variable to an environment variable, start
with the top-level property and separate every nested property with an
underscore (_).
  # For example, the property
"zeebe.broker.exporters.camundaExporter.args.index.numberOfShards" would be
converted to
"ZEEBE_BROKER_EXPORTERS_CAMUNDAEXPORTER_ARGS_INDEX_NUMBEROFSHARDS".
  #
  camundaExporter:
    className: io.camunda.exporter.CamundaExporter

    args:
      connect:
        type: elasticsearch
        url: http://localhost:9200
        clusterName: elasticsearch
        dateFormat: yyyy-MM-dd'T'HH:mm:ss.SSSZZ
        socketTimeout: 1000
        connectTimeout: 1000
        username: elastic
        password: changeme
        security:
          enabled: false
```

```
      certificatePath: /path/to/certificate
      verifyHostname: true
      selfSigned: false

  bulk:
    delay: 5
    size: 1000

  index:
    prefix:
    numberOfShards: 3
    numberOfReplicas: 0

  retention:
    enabled: false
    minimumAge: 30d
    policyName: camunda-retention-policy

  createSchema: true
```

Elasticsearch exporter

NOTE

Please refer to supported environments to find out which versions of Elasticsearch are supported in a Camunda 8 Self-Managed setup.

The Zeebe Elasticsearch exporter acts as a bridge between Zeebe and Elasticsearch by exporting records written to Zeebe streams as documents into several indices.

Concept

The exporter operates on the idea that it should perform as little as possible on the Zeebe side of things. In other words, you can think of the indexes into which the records are exported as a staging data warehouse. Any enrichment or transformation on the exported data should be performed by your own ETL jobs.

When configured to do so, the exporter will automatically create an index per record value type (see the value type in the Zeebe protocol). Each of these indexes has a corresponding pre-defined mapping to facilitate data ingestion for your own ETL jobs. You can find those as templates in the resources folder of the exporter's source code.

NOTE

The indexes are created as required, and will not be created twice if they already exist. However, once disabled, they will not be deleted (that is up to the administrator.)

Similarly, data is never deleted by the exporter, and must be deleted by the administrator when it is safe to do so. A retention policy can be configured to automatically delete data after a certain number of days.

Configuration

NOTE

As the exporter is packaged with Zeebe, it is not necessary to specify a `jarPath`.

The exporter can be enabled by configuring it with the `classpath` in the broker settings.

For example:

```
exporters:
  elasticsearch:
    className: io.camunda.zeebe.exporter.ElasticsearchExporter
    args:
    # Refer to the table below for the available args options
```

The exporter can be configured by providing `args`. The table below explains all the different options, and the default values for these options:

Option	Description	Default
url	Valid URLs as comma-separated string	`http://localhost:9200`
requestTimeoutMs	Request timeout (in ms) for the Elasticsearch. client	`30000`
index	Refer to Index for the index configuration	

Option	Description	Default
	options.	
bulk	Refer to Bulk for the bulk configuration options.	
retention	Refer to Retention for the retention configuration options	
authentication	Refer to Authentication for the authentication configuration options.	

- Index
- In most cases, you will not be interested in exporting every single record produced by a Zeebe cluster, but rather only a subset of them. This can also be configured to limit the kinds of records being exported (e.g. only events, no commands), and the value type of these records (e.g. only job and process values).

Option	Description	Default
prefix	This prefix will be appended to every index created by the exporter; must not contain _ (underscore).	zeebe-record
createTemplate	If `true` missing indexes will be created automatically.	`true`

Option	Description	Default
indexSuffixDatePattern	This suffix will be appended to every index created by the exporter; The pattern is based on the Java DateTimeFormater and supports the same syntax. This is useful when indexes should be created in a different interval, like hourly instead of daily.	`"yyyy-MM-dd'"`
numberOfShards	The number of shards used for each new record index created.	3
numberOfReplicas	The number of shard replicas used for each new record index created.	0
command	If `true` command records will be exported	`false`
event	If `true` event records will be exported	`true`
rejection	If `true` rejection records will be exported	`false`
checkpoint	If `true` records related to checkpoints will be exported	`false`
commandDistribution	If `true` records related to command distributions will be exported	`true`
decision	If `true` records related to decisions will be exported	`true`

Option	Description	Default
decisionEvaluation	If `true` records related to decision evaluations will be exported	`true`
decisionRequirements	If `true` records related to decisionRequirements will be exported	`true`
deployment	If `true` records related to deployments will be exported	`true`
deploymentDistribution	If `true` records related to deployment distributions will be exported	`true`
error	If `true` records related to errors will be exported	`true`
escalation	If `true` records related to escalations will be exported	`true`
form	If `true` records related to forms will be exported	`true`
incident	If `true` records related to incidents will be exported	`true`
job	If `true` records related to jobs will be exported	`true`
jobBatch	If `true` records related to job batches will be exported	`false`
message	If `true` records related to messages	`true`

Option	Description	Default
	will be exported	
messageSubscription	If `true` records related to message subscriptions will be exported	`true`
messageStartEventSubscription	If `true` records related to message start event subscriptions will be exported	`true`
process	If `true` records related to processes will be exported	`true`
processEvent	If `true` records related to process events will be exported	`false`
processInstance	If `true` records related to process instances will be exported	`true`
processInstanceBatch	If `true` records related to process instances batches will be exported	`false`
processInstanceCreation	If `true` records related to process instance creations will be exported	`true`
processInstanceMigration	If `true` records related to process instance migrations will be exported	`true`
processInstanceModification	If `true` records related to process instance modifications will be exported	`true`
processMessageSubscription	If `true` records related to process	`true`

Option	Description	Default
	message subscriptions will be exported	
resourceDeletion	If `true` records related to resource deletions will be exported	`true`
signal	If `true` records related to signals will be exported	`true`
signalSubscription	If `true` records related to signal subscriptions will be exported	`true`
timer	If `true` records related to timers will be exported	`true`
userTask	If `true` records related to user tasks will be exported	`true`
variable	If `true` records related to variables will be exported	`true`
variableDocument	If `true` records related to variable documents will be exported	`true`

- Bulk

To avoid too many expensive requests to the Elasticsearch cluster, the exporter performs batch updates by default. The size of the batch, along with how often it should be flushed (regardless of size) can be controlled by configuration.

Option	Description	Default
delay	Delay, in seconds, before force flush of the current batch. This ensures that even when we have low traffic of records, we still export every once in a while.	`5`
size	The amount of records a batch should have before we flush the batch	`1000`
memoryLimit	The size of the batch, in bytes, before we flush the batch	`10485760` (10 MB)

With the default configuration, the exporter will aggregate records and flush them to Elasticsearch:

1. When it has aggregated 1000 records.
2. When the batch memory size exceeds 10 MB.
3. Five seconds have elapsed since the last flush (regardless of how many records were aggregated).

- Retention

A retention policy can be set up to delete old data. When enabled, this creates an Index Lifecycle Management (ILM) Policy that deletes the data after the specified `minimumAge`. All index templates created by this exporter apply the created ILM Policy.

Option	Description	Default

Option	Description	Default
enabled	If `true` the ILM Policy is created and applied to the index templates	`false`
minimumAge	Specifies how old the data must be, before the data is deleted as a duration	`30d`
policyName	The name of the created and applied ILM policy	`zeebe-record-retention-policy`

NOTE

The duration can be specified in days `d`, hours `h`, minutes `m`, seconds `s`, milliseconds `ms`, and/or nanoseconds `nanos`.

- Authentication
- Providing these authentication options will enable Basic Authentication on the exporter.

Option	Description	Default
username	Username used to authenticate	N/A
password	Password used to authenticate	N/A

Example

Here is an example configuration of the exporter:

```
---
exporters:
  elasticsearch:
    # Elasticsearch Exporter ----------
    # An example configuration for the elasticsearch exporter:
    #
    # These setting can also be overridden using the environment variables
"ZEEBE_BROKER_EXPORTERS_ELASTICSEARCH_..."
    #

    className: io.camunda.zeebe.exporter.ElasticsearchExporter

    args:
      # A comma separated list of URLs pointing to the Elasticsearch
instances you wish to export to.
      # For example, if you want to connect to multiple nodes for
redundancy:
      # url: http://localhost:9200,http://localhost:9201
      url: http://localhost:9200

      bulk:
        delay: 5
        size: 1000
        memoryLimit: 10485760

      retention:
        enabled: true
        minimumAge: 30d
        policyName: zeebe-records-retention-policy

      authentication:
        username: elastic
        password: changeme

      index:
        prefix: zeebe-record
        createTemplate: true

        indexSuffixDatePattern: "yyyy-MM-dd"

        command: false
        event: true
        rejection: false

        commandDistribution: true
        decisionRequirements: true
        decision: true
        decisionEvaluation: true
        deployment: true
        deploymentDistribution: true
        error: true
        escalation: true
        form: true
        incident: true
        job: true
        jobBatch: false
        message: true
        messageStartSubscription: true
        messageSubscription: true
        process: true
        processEvent: false
```

```
        processInstance: true
        processInstanceCreation: true
        processInstanceMigration: true
        processInstanceModification: true
        processMessageSubscription: true
        resourceDeletion: true
        signal: true
        signalSubscription: true
        timer: true
        userTask: true
        variable: true
        variableDocument: true
```

OpenSearch exporter

NOTE

Please refer to Supported Environments to find out which versions of OpenSearch are supported in a Camunda 8 Self-Managed setup.

The Zeebe OpenSearch Exporter acts as a bridge between Zeebe and OpenSearch by exporting records written to Zeebe streams as documents into several indices.

Concept

The exporter operates on the idea that it should perform as little as possible on the Zeebe side of things. In other words, you can think of the indexes into which the records are exported as a staging data warehouse. Any enrichment or transformation on the exported data should be performed by your own ETL jobs.

When configured to do so, the exporter will automatically create an index per record value type (see the value type in the Zeebe protocol). Each of these indexes has a corresponding pre-defined mapping to facilitate data ingestion for your own ETL jobs. You can find those as templates in this module's resources folder.

NOTE

The indexes are created as required, and will not be created twice if they already exist. However, once disabled, they will not be deleted (that is up to the administrator). Similarly, data is never deleted by the exporter, and must be deleted by the administrator when it is safe to do so.

Configuration

NOTE

As the exporter is packaged with Zeebe, it is not necessary to specify a `jarPath`.

The exporter can be enabled by configuring it with the classpath in the broker settings.

For example:

```
exporters:
  opensearch:
    className: io.camunda.zeebe.exporter.opensearch.OpensearchExporter
    args:
    # Refer to the table below for the available args options
```

The exporter can be configured by providing `args`. The table below explains all the different options, and the default values for these options:

Option	Description	Default
url	Valid URLs as comma-separated string	`http://localhost:9200`
requestTimeoutMs	Request timeout (in ms) for the OpenSearch client.	`30000`
index	Refer to Index for the index configuration options.	
bulk	Refer to Bulk for the bulk configuration options.	
retention	Refer to Retention for the retention	

Option	Description	Default
	configuration options.	
authentication	Refer to Authentication for the authentication configuration options.	
aws	Refer to AWS for the AWS configuration options.	

- Index
- In most cases, you will not be interested in exporting every single record produced by a Zeebe cluster, but rather only a subset of them. This can also be configured to limit the kinds of records exported (e.g. only events, no commands), and the value type of these records (e.g. only job and process values).

Option	Description	Default
prefix	This prefix will be appended to every index created by the exporter; must not contain _ (underscore).	zeebe-record
createTemplate	If `true` missing indexes will be created automatically.	`true`
numberOfShards	The number of shards used for each new record index created.	3
numberOfReplicas	The number of shard replicas used for	0

Option	Description	Default
	each new record index created.	
command	If `true` command records will be exported	`false`
event	If `true` event records will be exported	`true`
rejection	If `true` rejection records will be exported	`false`
checkpoint	If `true` records related to checkpoints will be exported	`false`
commandDistribution	If `true` records related to command distributions will be exported	`true`
decision	If `true` records related to decisions will be exported	`true`
decisionEvaluation	If `true` records related to decision evaluations will be exported	`true`
decisionRequirements	If `true` records related to decisionRequirements will be exported	`true`
deployment	If `true` records related to deployments will be exported	`true`
deploymentDistribution	If `true` records related to deployment distributions will be exported	`true`

Option	Description	Default
error	If `true` records related to errors will be exported	`true`
escalation	If `true` records related to escalations will be exported	`true`
form	If `true` records related to forms will be exported	`true`
incident	If `true` records related to incidents will be exported	`true`
job	If `true` records related to jobs will be exported	`true`
jobBatch	If `true` records related to job batches will be exported	`false`
message	If `true` records related to messages will be exported	`true`
messageSubscription	If `true` records related to message subscriptions will be exported	`true`
messageStartEventSubscription	If `true` records related to message start event subscriptions will be exported	`true`
process	If `true` records related to processes will be exported	`true`

Option	Description	Default
processEvent	If `true` records related to process events will be exported	`false`
processInstance	If `true` records related to process instances will be exported	`true`
processInstanceBatch	If `true` records related to process instances batches will be exported	`false`
processInstanceCreation	If `true` records related to process instance creations will be exported	`true`
processInstanceMigration	If `true` records related to process instance migrations will be exported	`true`
processInstanceModification	If `true` records related to process instance modifications will be exported	`true`
processMessageSubscription	If `true` records related to process message subscriptions will be exported	`true`
resourceDeletion	If `true` records related to resource deletions will be exported	`true`
signal	If `true` records related to signals will be exported	`true`
signalSubscription	If `true` records related to signal subscriptions will be exported	`true`

Option	Description	Default
timer	If `true` records related to timers will be exported	`true`
userTask	If `true` records related to user tasks will be exported	`true`
variable	If `true` records related to variables will be exported	`true`
variableDocument	If `true` records related to variable documents will be exported	`true`

- Bulk

To avoid too many expensive requests to the OpenSearch cluster, the exporter performs batch updates by default. The size of the batch, along with how often it should be flushed (regardless of size) can be controlled by configuration.

Option	Description	Default
delay	Delay, in seconds, before force flush of the current batch. This ensures that even when we have low traffic of records, we still export every once in a while.	`5`
size	The amount of records a batch should have before we flush the batch.	`1000`

Option	Description	Default
memoryLimit	The size of the batch, in bytes, before we flush the batch.	`10485760` (10 MB)

With the default configuration, the exporter would aggregate records and flush them to OpenSearch either:

1. When it has aggregated 1000 records.
2. When the batch memory size exceeds 10 MB.
3. Five seconds have elapsed since the last flush (regardless of how many records were aggregated).

- Retention
- A retention policy can be set up to delete old data. When enabled, this creates an Index State Management (ISM) policy that deletes the data after the specified `minimumAge`. All index templates created by this exporter apply the created ISM policy.

Option	Description	Default
enabled	If `true` the ISM policy is created and applied to the index templates.	`false`
minimumAge	Specifies how old the data must be, before the data is deleted as a duration.	`30d`

Option	Description	Default
policyName	The name of the created and applied ISM policy.	`zeebe-record-retention-policy`
policyDescription	The description of the created and applied ISM policy.	`Zeebe record retention policy`

- NOTE
- The duration can be specified in days `d`, hours `h`, minutes `m`, seconds `s`, milliseconds `ms`, and/or nanoseconds `nanos`.

- Authentication

Providing these authentication options will enable Basic Authentication on the exporter.

Option	Description	Default
username	Username used to authenticate	N/A
password	Password used to authenticate	N/A

- AWS
- When running OpenSearch in AWS, you may require requests to be signed. By enabling AWS in the configurations, a request interceptor will be added to the exporter. This interceptor will take care of signing the requests.
- Signing requests requires credentials. These credentials are not directly configurable in the exporter. Instead, they are resolved by following the Default Credential Provider Chain.

Option	Description	Default
enabled	Enables AWS request signing	`false`
serviceName	AWS' name of the service to where requests are made. For OpenSearch this should be `es`	`es`
region	The region this exporter is running in	The value of the `AWS_REGION` environment variable

Example

Here is an example configuration of the exporter:

```
---
exporters:
  opensearch:
    # Opensearch Exporter ----------
    # An example configuration for the opensearch exporter:
    #
    # These setting can also be overridden using the environment variables
"ZEEBE_BROKER_EXPORTERS_OPENSEARCH_..."

    className: io.camunda.zeebe.exporter.opensearch.OpensearchExporter

    args:
      # A comma separated list of URLs pointing to the Opensearch instances
you wish to export to.
      # For example, if you want to connect to multiple nodes for
redundancy:
      # url: http://localhost:9200,http://localhost:9201
      url: http://localhost:9200

      bulk:
        delay: 5
        size: 1000
        memoryLimit: 10485760

      retention:
        enabled: true
        minimumAge: 30d
        policyName: zeebe-records-retention-policy
        policyDescription: Zeebe records retention policy
```

```yaml
authentication:
  username: opensearch
  password: changeme

aws:
  enabled: true
  serviceName: es
  region: eu-west-1

index:
  prefix: zeebe-record
  createTemplate: true

  command: false
  event: true
  rejection: false

  commandDistribution: true
  decisionRequirements: true
  decision: true
  decisionEvaluation: true
  deployment: true
  deploymentDistribution: true
  error: true
  escalation: true
  form: true
  incident: true
  job: true
  jobBatch: false
  message: true
  messageStartSubscription: true
  messageSubscription: true
  process: true
  processEvent: false
  processInstance: true
  processInstanceCreation: true
  processInstanceMigration: true
  processInstanceModification: true
  processMessageSubscription: true
  resourceDeletion: true
  signal: true
  signalSubscription: true
  timer: true
  userTask: true
  variable: true
  variableDocument: true
```